

Session 3 — JWT Authentication in API Gateway

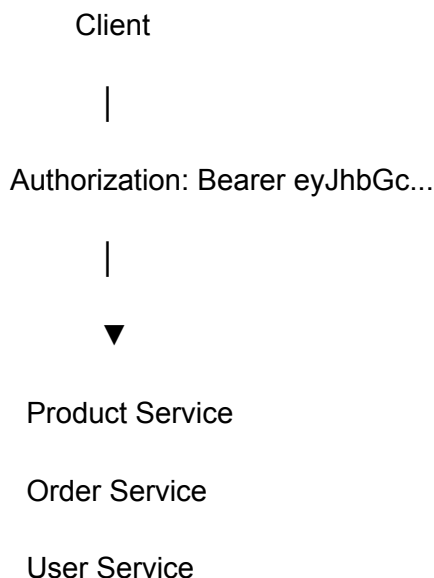
Learning Objectives

By the end of this session, students will be able to:

- Understand why JWT validation should happen in the API Gateway.
 - Understand the concept of **Security Boundary**.
 - Differentiate between **Public** and **Protected** endpoints.
 - Configure **Public Routes** using **HTTP Method + Path**.
 - Implement JWT validation using a **GlobalFilter**.
 - Enrich downstream requests with authenticated user information.
 - Understand why downstream services do **NOT** validate JWT again.
-

Why do we validate JWT in the Gateway?

Suppose we have three microservices.



Without an API Gateway, every service must:

- Read the Authorization header
- Validate the JWT signature
- Check expiration

- Extract claims
- Reject invalid tokens

If we have:

10 Microservices

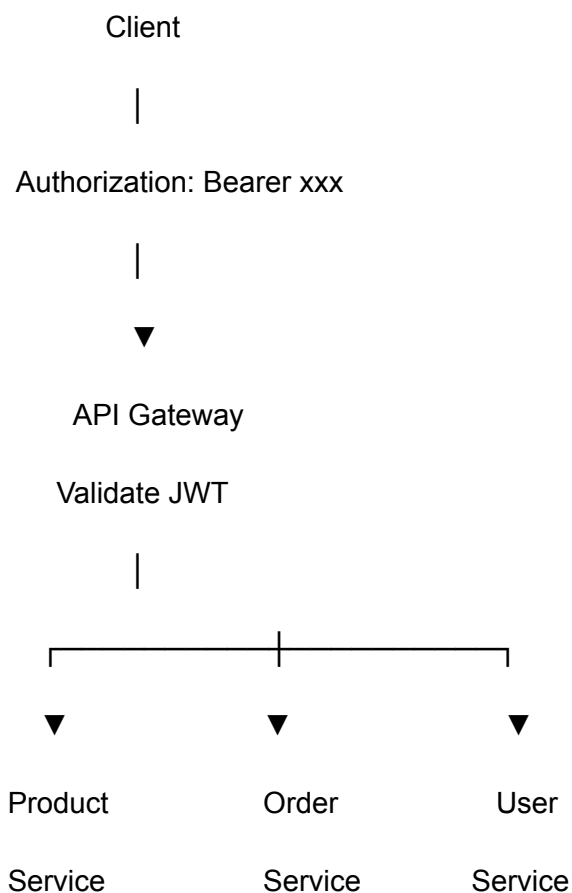
Then we duplicate the exact same authentication code:

10 Times

This violates the **DRY (Don't Repeat Yourself)** principle.

The Solution

Instead, place a Gateway in front of all services.



Now only **one component** is responsible for authentication.

Security Boundary

This is the most important concept of this session.

The Gateway validates the JWT. Downstream services trust the Gateway.

The services no longer care about JWT.

Instead they trust the Gateway.

The Gateway becomes the **Security Boundary** of the system.

Request Lifecycle

When a request reaches the Gateway:

Client

↓

Authorization: Bearer eyJhbGc...

↓

Gateway

The Gateway performs:

1. Check whether the endpoint is public.
 2. Read Authorization header.
 3. Validate JWT.
 4. Extract user claims.
 5. Add user information as headers.
 6. Forward request.
-

Public Routes

Some endpoints should be accessible without authentication.

Examples:

GET /api/products

POST /api/auth/login

POST /api/auth/register

GET /actuator/health

These endpoints are called:

Public Routes

Protected Routes

Examples:

POST /api/products

PUT /api/products/5

DELETE /api/products/5

GET /api/orders

These require authentication.

Why Method + Path?

Many beginners implement:

```
private boolean isPublicRoute(String path) {  
    return path.startsWith("/api/products");  
}
```

Looks correct...

But consider these requests:

Request	Result
GET /api/products	Public 
POST /api/products	Public  (Wrong!)

The path is identical.

The HTTP Method is different.

Therefore we must validate:

HTTP Method

+

URL Path

instead of only the path.

Representing a Public Route

We define a small record.

```
private record PublicRoute(  
    HttpMethod method,  
    PathPattern pattern  
) {  
}
```

Now every public endpoint contains:

- HTTP Method
 - URL Pattern
-

Defining Public Routes

```
private static final PathPatternParser PARSER = new PathPatternParser();
```

```
private static final List<PublicRoute> PUBLIC_ROUTES = List.of(  
    new PublicRoute(  
        HttpMethod.GET,
```

```
        PARSER.parse("/api/products")
    ),
    new PublicRoute(
        HttpMethod.POST,
        PARSER.parse("/api/auth/login")
    ),
    new PublicRoute(
        HttpMethod.POST,
        PARSER.parse("/api/auth/register")
    ),
    new PublicRoute(
        HttpMethod.GET,
        PARSER.parse("/actuator/health")
    )
);
```

Notice:

GET /api/products

is public.

But

POST /api/products

is NOT.

الأفضل أيضاً

داخل الكود Hardcoded List بدلاً من

اجعلها داخل

application.yml

مثل

security:

public-routes:

- method: GET

path: /api/v1/products/**

- method: GET

path: /actuator/health

- method: GET

path: /actuator/info

ثم

```
@ConfigurationProperties(prefix = "security")
```

```
public class SecurityProperties {
```

```
    private List<PublicRoute> publicRoutes;
```

```
}
```

Compile جديدة لا يحتاج إعادة Route وهذا يجعل إضافة

وهذا أقرب لما يُطبق في المشاريع الحقيقية

JWT Authentication GlobalFilter

Now let's put everything together.

```
package com.microservices.pro.apigateway.filter;
```

```
import com.microservices.pro.apigateway.security.JwtUtil;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import lombok.RequiredArgsConstructor;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.core.Ordered;
import org.springframework.core.io.buffer.DataBuffer;
import org.springframework.http.*;
import org.springframework.http.server.PathContainer;
import org.springframework.http.server.reactive.ServerHttpRequest;
import org.springframework.http.server.reactive.ServerHttpResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import org.springframework.web.util.pattern.PathPattern;
import org.springframework.web.util.pattern.PathPatternParser;
import reactor.core.publisher.Mono;
```

```

import java.nio.charset.StandardCharsets;
import java.util.List;

@Component
@RequiredArgsConstructor
public class JwtAuthFilter implements GlobalFilter, Ordered {

    private final JwtUtil jwtUtil;

    private static final PathPatternParser PARSER =
        new PathPatternParser();

    private record PublicRoute(HttpMethod method,
                               PathPattern pattern) {
    }

    private static final List<PublicRoute> PUBLIC_ROUTES = List.of(

        new PublicRoute(
            HttpMethod.GET,
            PARSER.parse("/api/products")
        ),

        new PublicRoute(
            HttpMethod.POST,
            PARSER.parse("/api/auth/login")
        ),

        new PublicRoute(
            HttpMethod.POST,
            PARSER.parse("/api/auth/register")
        ),

        new PublicRoute(
            HttpMethod.GET,
            PARSER.parse("/actuator/health")
        )
    );

    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
                            GatewayFilterChain chain) {

        ServerHttpRequest request =
            exchange.getRequest();

        String path =
            request.getURI().getPath();
    }

```

```

    HttpMethod method =
        request.getMethod();

    // -----
    // Step 1
    // Skip authentication for Public Routes
    // -----

    if (isPublicRoute(method, path)) {

        return chain.filter(exchange);

    }

    // -----
    // Step 2
    // Read Authorization Header
    // -----

    String authHeader =
        request.getHeaders()
            .getFirst(HttpHeaders.AUTHORIZATION);

    if (authHeader == null ||
        !authHeader.startsWith("Bearer ")) {

        return unauthorized(
            exchange,
            "Missing or invalid Authorization header"
        );

    }

    // -----
    // Step 3
    // Validate JWT
    // -----

    String token =
        authHeader.substring(7);

    try {

        Claims claims =
            jwtUtil.validateToken(token);

        // -----

```

```

// Step 4
// Header Enrichment
// -----

ServerHttpRequest enrichedRequest =
    request.mutate()
        .headers(headers -> {

            headers.remove("X-User-Id");
            headers.remove("X-User-Role");

            headers.add(
                "X-User-Id",
                claims.getSubject());

            headers.add(
                "X-User-Role",
                claims.get("role",
                    String.class));

        })
        .build();

return chain.filter(

    exchange.mutate()
        .request(enrichedRequest)
        .build()

);

}

catch (JwtException ex) {

    return unauthorized(
        exchange,
        "Invalid or expired JWT"
    );

}

}

private boolean isPublicRoute(HttpMethod method,
    String path) {

    PathContainer pathContainer =

```

```

        PathContainer.parsePath(path);

return PUBLIC_ROUTES.stream()

        .anyMatch(route ->

            route.method() == method

                &&

                route.pattern()
                    .matches(pathContainer)

        );
}

private Mono<Void> unauthorized(

    ServerWebExchange exchange,

    String message

){

    ServerHttpResponse response =
        exchange.getResponse();

    response.setStatusCode(
        HttpStatus.UNAUTHORIZED);

    response.getHeaders().setContentType(
        MediaType.APPLICATION_JSON);

    String body = ""
        {
            "status":401,
            "error":"Unauthorized",
            "message":"%s"
        }
        "" formatted(message);

    DataBuffer buffer =
        response.bufferFactory()
            .wrap(body.getBytes(
                StandardCharsets.UTF_8));

    return response.writeWith(

```

```
        Mono.just(buffer));  
  
    }  
  
    @Override  
    public int getOrder() {  
  
        return Ordered.HIGHEST_PRECEDENCE + 1;  
  
    }  
  
}
```

Understanding the Filter

Step 1 — Check Public Routes

The filter first checks whether the request is public.

GET /api/products

↓

Public

↓

Forward immediately

No JWT validation.

But

POST /api/products

↓

Protected

↓

JWT validation is required.

Step 2 — Read Authorization Header

Expected format:

Authorization: Bearer eyJhbGc...

If the header is missing:

401 Unauthorized

is returned immediately.

The request never reaches the Product Service.

Step 3 — Validate JWT

Claims claims =

```
jwtUtil.validateToken(token);
```

If the token is:

- Expired
- Invalid
- Tampered

The Gateway returns:

401 Unauthorized

Header Enrichment

After successful validation,

the Gateway extracts claims.

Example JWT:

```
{  
  "sub": "25",  
  "role": "ADMIN"  
}
```

The Gateway converts them into headers.

X-User-Id : 25

X-User-Role : ADMIN

Then forwards the request.

Why Remove Existing Headers?

Imagine a malicious client sends:

X-User-Role: ADMIN

If the Gateway simply forwards it,
the attacker may gain unauthorized access.

Therefore the Gateway always does:

```
headers.remove("X-User-Id");  
headers.remove("X-User-Role");
```

Then adds trusted values extracted from the validated JWT.

```
headers.add("X-User-Id", claims.getSubject());
```

```
headers.add("X-User-Role",  
    claims.get("role", String.class));
```

This guarantees that downstream services only receive trusted identity information.

Product Service

هكذا Product Service سيصل الطلب إلى:

GET /api/products/1

Headers

X-User-Id: 123

X-User-Role: ADMIN

Notice that the Product Service never validates JWT.

Instead it simply reads the headers.

@RequestHeader

```
@GetMapping("/{id}")
public ResponseEntity<Product> getProduct(

    @PathVariable Long id,

    @RequestHeader("X-User-Id")
    String userId,

    @RequestHeader("X-User-Role")
    String role

){

    log.info("User ID : {}", userId);

    log.info("Role : {}", role);

    return ResponseEntity.ok(...);

}
```

HttpServletRequest

Headers. إذا أردت قراءة عدد كبير من الـ

```
@GetMapping("/{id}")
public ResponseEntity<Product> getById(

    @PathVariable Long id,

    HttpServletRequest request

){

    String userId = request.getHeader("X-User-Id");

    String role = request.getHeader("X-User-Role");

    log.info(userId);

    log.info(role);

}
```

```

    return ResponseEntity.ok(...);
}

```

HttpHeaders

```

@GetMapping("/{id}")
public ResponseEntity<Product> getById(
    @PathVariable Long id,
    @RequestHeader HttpHeaders headers
){
    String userId = headers.getFirst("X-User-Id");
    String role = headers.getFirst("X-User-Role");
    return ResponseEntity.ok(...);
}

```

The service focuses only on business logic.

Authentication has already been completed by the Gateway.

Complete Request Flow

Client

Authorization: Bearer eyJhbGc...

|



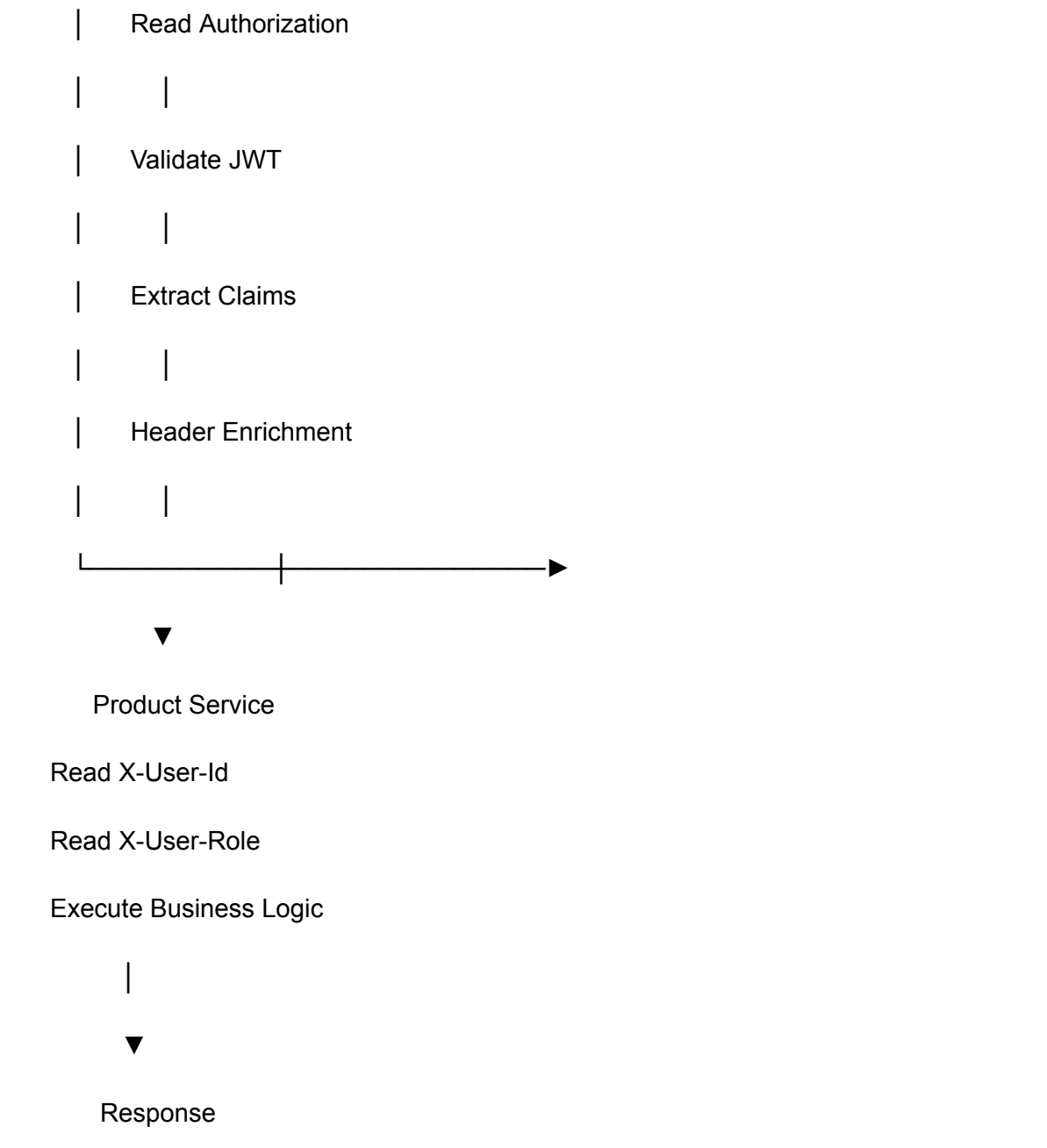
API Gateway

Is Public Route?

YES NO

|

|



Summary

Concept	Description
Security Boundary	JWT is validated only in the API Gateway.
Public Route	Defined using HTTP Method + Path .
Protected Route	Requires a valid JWT before forwarding.
JWT Validation	Performed once in the Gateway.

Header Enrichment	Gateway extracts claims and forwards them as trusted headers.
Product Service	Reads trusted headers and executes business logic without re-validating JWT.

Key Takeaway: The API Gateway is responsible for authentication. Downstream microservices trust the Gateway and focus solely on business functionality. This separation of concerns reduces code duplication, centralizes security, and simplifies the design of the entire microservices platform.

Part 2 — Rate Limiting with Redis

Learning Objectives

بنهاية هذا الدرس سيكون الطالب قادرًا على:

- فهم لماذا نستخدم Rate Limiting.
- فهم فكرة Token Bucket Algorithm.
- معرفة معنى Token.
- فهم الفرق بين `replenishRate` و `burstCapacity`.
- معرفة لماذا يستخدم Redis.
- توقع سلوك النظام عند زيادة عدد الطلبات.

لماذا نحتاج Rate Limiting؟

تخيل أن لدينا API

POST /api/orders

وأحد المستخدمين أرسل

10000 Requests

في ثانية واحدة.

ما الذي سيحدث؟

- ارتفاع CPU
- استهلاك الذاكرة
- ضغط على قاعدة البيانات
- بطء النظام
- وربما يتوقف بالكامل (Denial of Service)

لذلك نضع Rate Limiter أمام الخدمات.

Client

|



API Gateway

Rate Limiter



Order Service

أي طلب يتجاوز الحد يتم رفضه قبل الوصول إلى الخدمة.

الفكرة الأساسية

نتخيل أن لكل مستخدم **Bucket** (دلو).

داخل هذا الدلو توجد Tokens.

كل Request يحتاج Token واحد.

Bucket



إذا وجد Token



يسمح بالطلب.

إذا انتهت جميع الـ Tokens



يرجع Gateway

429 Too Many Requests

ما هو Token؟

الـ Token هنا ليس JWT.

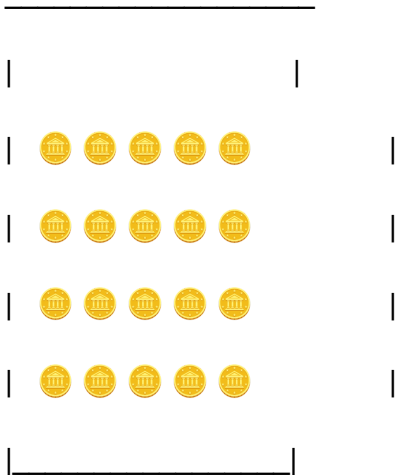
لدينا نوعان مختلفان تمامًا:

الاستخدام	Token
Authentication	JWT Token
Rate Limiting	Bucket Token
Token Bucket مجرد عداد (Counter).	

Token Bucket Algorithm

تخيل هذا الدلو.

Capacity = 20



20 Tokens

كل Request يأخذ Token واحد.

إذا جاء 3 Requests

Before

20 Tokens



Request



19

↓

Request

↓

18

↓

Request

↓

17

ما هو replenishRate؟

هذه الخاصية تعني:

كم Token نضيف كل ثانية إلى الدلو.

مثلاً

replenishRate: 10

تعني

كل ثانية

↓

أضف

10 Tokens

إلى الدلو.

مثال

لدينا

Bucket = 5 Tokens

وبعد مرور ثانية

+10 Tokens

فيصبح

15 Tokens

بعد ثانية أخرى

+10

يصبح

25

لكن...

ماذا لو تجاوز السعة؟

هنا يأتي دور

burstCapacity

ما هو burstCapacity؟

هو

أقصى عدد Tokens يمكن أن يحتفظ به الدلو.

مثلاً

burstCapacity: 20

يعني

حتى لو أضفنا Tokens

لن يزيد العدد عن

20

مثال

لدينا

18 Tokens

بعد ثانية

+10

المفروض يصبح

28

لكن

الحد الأقصى

20

إذن يصبح

فقط 20

وليس

28

إذن نعم، إجابتك صحيحة.

في المثال:

replenishRate: 10

burstCapacity: 20

كل ثانية يضيف النظام **10 Tokens**، لكن إذا تجاوز العدد **20** فإنه يبقى عند **20**.

أي أن الدلو لا يمتلئ أكثر من سعته.

ماذا يعني **Steady State**؟

Steady State تعني:

عندما يعمل النظام لفترة طويلة وبشكل مستمر.

أي بعد انتهاء أي **Burst** أولي، يصبح النظام في حالة مستقرة.

مثلاً

replenishRate = 10

يعني

كل ثانية

↓

يضاف

Tokens 10

إنن

يمكن استهلاك

Requests 10

كل ثانية

بشكل مستمر.

ولهذا نقول

Steady State = 10 Requests / Second

مثال

إذا أرسل المستخدم

10 Requests

كل ثانية

فسيحذف التالي

Second 1

+10 Tokens

↓

Consume 10

↓

0 Used

Second 2

+10

↓

Consume 10

↓

0 Used

Second 3

+10

↓

Consume 10

ويستمر النظام إلى ما لا نهاية.

ماذا يعني Burst؟

Burst يعني

ارتفاع مفاجئ في عدد الطلبات.

مثلاً

بدلاً من

Requests 10

جاء فجأة

Requests 20

في نفس اللحظة.

إذا كان

burstCapacity = 20

وكان الدلو ممتلئاً

20 Tokens

فيمكنه استيعاب

20 Requests

دفعة واحدة.

ثم يصبح

0 Tokens

بعد ثانية

يضيف

10 Tokens

فيصبح

10

إذا حاول العميل إرسال

Request 20

مرة أخرى

فلن ينجح.

سيسمح فقط بـ

10

ثم

يرجع

429 Too Many Requests

للباقى الطلبات.

هل burstCapacity يعني عدد الطلبات في الثانية؟

لا.

وهذه من أكثر الأخطاء شيوعًا.

burstCapacity لا يعني "طلبات في الثانية"، وإنما يعني أقصى عدد من الـ **Tokens** يمكن تخزينه.

إذا كان الدلو ممتلئًا، يمكن استهلاك جميع هذه الـ **Tokens** فورًا في جزء من الثانية، لذا قد يسمح بأكثر من **replenishRate** خلال لحظة قصيرة.

مثال زمني كامل

نفترض:

$\text{replenishRate} = 10$

$\text{burstCapacity} = 20$

في البداية

$\text{Bucket} = 20$

الثانية 0

$\text{Bucket} = 20$

يرسل العميل

15 Requests

النظام يسمح بها جميعًا.

يبقى

5 Tokens

الثانية 1

النظام يضيف

+10

فيصبح

15 Tokens

يرسل العميل

12 Requests

يبقى

3 Tokens

الثانية 2

+10

يصبح

13

يرسل العميل

25 Requests

يسمح فقط بـ

13

ويرفض

12

بـ

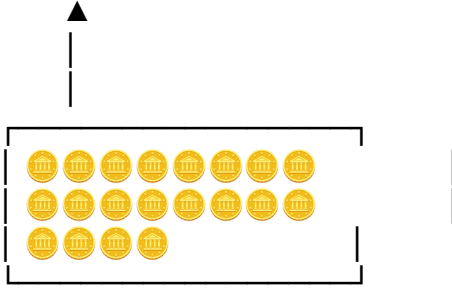
429 Too Many Requests

الرسم النهائي

Bucket

Maximum Capacity = 20

+10 every second
(replenishRate)



Each Request



Consumes 1 Token



No Token?



429 Too Many Requests

ملخص المفاهيم

معناها	الخاصية
وحدة سماح (Permit) لطلب واحد، ولا علاقة لها بـ JWT.	Token
دلو افتراضي يحتفظ بعدد من الـ Tokens لكل عميل أو مفتاح (Key).	Token Bucket
عدد الـ Tokens التي تُضاف إلى الدلو كل ثانية.	replenishRate
الحد الأقصى لعدد الـ Tokens التي يمكن تخزينها في الدلو.	burstCapacity
معدل الطلبات الذي يمكن دعمه باستمرار، ويساوي تقريباً قيمة replenishRate طالما لا توجد Tokens مخزنة إضافية.	Steady State
اندفاع مؤقت من الطلبات يمكن استيعابه باستخدام الـ Tokens المخزنة حتى الوصول إلى burstCapacity .	Burst
يُرجع عندما لا يتبقى أي Token يسمح بتمرير الطلب.	Too Many 429 Requests
قاعدة سهلة للحفظ:	
• replenishRate = سرعة إعادة تعبئة الدلو (Tokens/Second).	

- **burstCapacity** = حجم الدلو (Maximum Tokens Stored).
- **Steady State** = المعدل المستمر الذي يستطيع النظام تحمله على المدى الطويل.
- **Burst** = استخدام الرصيد المخزن في الدلو للسماح بعدد كبير من الطلبات في وقت قصير.

Part 3 — Why Redis?

Learning Objectives

بنهاية هذا الجزء سيكون الطالب قادرًا على:

- فهم لماذا يحتاج Rate Limiter إلى Redis.
- معرفة ما الذي يتم تخزينه داخل Redis.
- فهم معنى **per client key**.
- فهم دور **KeyResolver**.
- معرفة ماذا يحدث إذا توقف Redis.
- استعراض القيم المخزنة داخل Redis عمليًا.

لماذا نحتاج Redis؟

لنفترض أن لدينا Gateway.

Client

|



API Gateway

ولدينا إعدادات:

replenishRate: 10

burstCapacity: 20

السؤال:

كيف يعرف الـ Gateway أن هذا المستخدم استهلك 15 Token بالفعل؟

أين سيحفظ العدد؟

لو لم يكن لدينا Redis...

كل Request سيكون مستقلاً.

Request 1



Gateway



20 Tokens

نعم

Request 2



Gateway



20 Tokens

نعم

Request 3



Gateway



20 Tokens

أي أن العداد يعود للصفر في كل مرة.

وبالتالي لن يعمل Rate Limiter إطلاقاً.

إذن أين نحفظ العداد؟

في Redis.

لهذا نضيف

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

ثم

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
```

ما الذي يخزنه Redis؟

ليس الطلبات.

وليس JWT.

وليس الـ Responses.

إنما يخزن فقط

عدد الـ **Tokens** المتبقية

لكل Client.

مثال

لدينا ثلاثة مستخدمين.

Ali

Ahmed

Sara

كل واحد لديه Bucket مستقل.

Ali

20 Tokens

Ahmed

20 Tokens

Sara

20 Tokens

Ali أرسل 8 Requests.

Redis يحدّث القيمة.

Ali

12 Tokens

Ahmed لم يرسل شيئاً.

Ahmed

20 Tokens

Sara أرسلت 19 Requests.

Sara

1 Token

إنّ Redis يحتفظ بحالة كل Bucket.

Redis stores the token count per client key

هذه الجملة تعني

Redis يحتفظ بعدد الـ Tokens لكل مفتاح (Key) يمثل عميلًا معينًا.

لاحظ كلمة

per client key

وليس

per server

ولا

per application

ما هو Client Key؟

هنا يأتي دور

key-resolver

key-resolver: "#{@ipKeyResolver}"

هذا يعني

كيف نميز المستخدمين عن بعضهم؟

مثلاً

Ali

عنوانه

192.168.1.5

Ahmed

192.168.1.20

إذن الـ Key يكون

192.168.1.5

وآخر

192.168.1.20

فيصبح لكل IP Bucket مستقل.

ماذا يخزن Redis؟

مثلاً

Key

192.168.1.5

↓

Remaining Tokens = 7

ومستخدم آخر

Key

192.168.1.20

↓

Remaining Tokens = 19

إذن

Redis عبارة عن جدول.

Remaining Tokens	Key
7	192.168.1.5
19	192.168.1.20
13	192.168.1.40

لماذا Redis وليس Memory؟

تخيل أن لدينا Gateway واحد.

Gateway

↓

Memory

سيعمل.

لكن ماذا لو أصبح لدينا

Gateway 1

Gateway 2

Gateway 3

كل واحد لديه Memory مختلفة.

سيصبح

Gateway1

Ali = 5 Tokens

بينما

Gateway2

Ali = 20 Tokens

وهذا خطأ.

أما Redis

Gateway 1

↓

Redis

↑

Gateway 2

↑

Gateway 3

جميعها ترى نفس البيانات.

RequestRateLimiter

filters:

- name: RequestRateLimiter

args:

redis-rate-limiter.replenishRate: 10

redis-rate-limiter.burstCapacity: 20

redis-rate-limiter.requestedTokens: 1

key-resolver: "#{@ipKeyResolver}"

requestedTokens

كل Request يستهلك

requestedTokens: 1

أي

1 Request

↓

1 Token

يمكن أيضًا جعله

requestedTokens: 5

أي

كل Request

↓

5 Tokens يستهلك

وهذا يستخدم أحياناً للعمليات المكلفة جداً.

ماذا يحدث إذا توقف Redis؟

السؤال المهم

Redis Down

كيف سيتحقق Gateway من عدد الـ Tokens؟

لا يستطيع.

السلوك الافتراضي في Spring Cloud Gateway هو

Fail Open

أي

Redis Down

↓

Disable Rate Limiting

↓

Allow Requests

لماذا اختار Spring هذا السلوك؟

لنفترض

لدينا

بنك

و Redis توقف لمدة دقيقة.

هل الأفضل

أن يتوقف الموقع بالكامل؟

أم يستمر الموقع ويعمل بدون Rate Limiting لمدة دقيقة؟

اختار Spring

Availability

بدلاً من

Strict Rate Limiting

وهذا يسمى

Fail Open

هناك أيضاً أسلوب آخر

Fail Closed

أي

Redis Down

↓

Reject Everything

هذا أكثر أماناً

لكن

قد يمنع جميع العملاء الحقيقيين من استخدام النظام.

كيف أشاهد القيم داخل Redis؟

إذا كنت تشغل Redis باستخدام Docker Compose.

مثلاً

services:

redis:

image: redis:7

ports:

- "6379:6379"

أدخل إلى الـ Container

docker ps

ستجد مثلاً

redis

ثم

docker exec -it redis redis-cli

الآن أنت داخل Redis.

اعرض جميع المفاتيح

KEYS *

أو (أفضل في الأنظمة الكبيرة)

SCAN 0

سترى مفاتيح مثل

request_rate_limiter.{192.168.1.5}

أو أسماء مشابهة، وقد تختلف الصيغة قليلاً حسب إصدار Spring Cloud Gateway.

اعرض قيمة المفتاح

GET request_rate_limiter.{192.168.1.5}

لكن هنا توجد ملاحظة مهمة.

ملاحظة مهمة جداً

في الحقيقة، **Spring Cloud Gateway** لا يخزن القيم كسلسلة نصية (**String**) يمكن قراءتها باستخدام **GET**.

هو يستخدم **Lua Scripts** ويخزن بيانات الـ Bucket في أكثر من Key (وأحياناً باستخدام أنواع بيانات مختلفة أو مفاتيح داخلية)، لذلك لن تجد قيمة مثل:

Remaining Tokens = 7

بشكل مباشر.

بدلاً من ذلك ستجد مفاتيح داخلية يديرها Spring و Redis معاً.

Redis يحتفظ بحالة (State) الـ Token Bucket لكل Client Key، بينما تفاصيل التخزين الداخلية يديرها **Spring Cloud Gateway** باستخدام **Lua Scripts** لتحقيق تحديثات ذرية (Atomic Operations) وأمنة حتى مع وجود عدة Gateways تعمل بالتوازي.

الرسم النهائي

Client

|



API Gateway

|

IP = 192.168.1.5

|



Redis

Key: 192.168.1.5

Remaining Tokens = 7

|



Product Service

ملخص الدرس

المفهوم	الوظيفة
Redis	يخزن حالة الـ Token Bucket لكل عميل بحيث لا تضيق بين الطلبات أو بين عدة Gateways.
Client Key	المفتاح الذي يميز كل عميل (مثل IP أو User ID)، ولكل مفتاح Bucket مستقل.
KeyResolver	يحدد كيف يتم إنشاء الـ Client Key الذي سيستخدمه الـ Rate Limiter.
requestedTokens	عدد الـ Tokens التي يستهلكها كل Request (الافتراضي 1).
Fail Open	إذا تعطل Redis، يمرر Gateway الطلبات بدون Rate Limiting للحفاظ على توفر الخدمة.
Redis CLI	يمكن استخدام <code>docker exec -it <container-name> redis-cli</code> ثم <code>SCAN 0</code> لاستعراض المفاتيح، لكن تفاصيل قيم الـ Rate Limiter تُدار داخليًا بواسطة Spring Cloud Gateway باستخدام Lua Scripts وليست قيمة نصية بسيطة يمكن قراءتها بـ <code>GET</code> .